

Sorting Techniques & Time Analysis

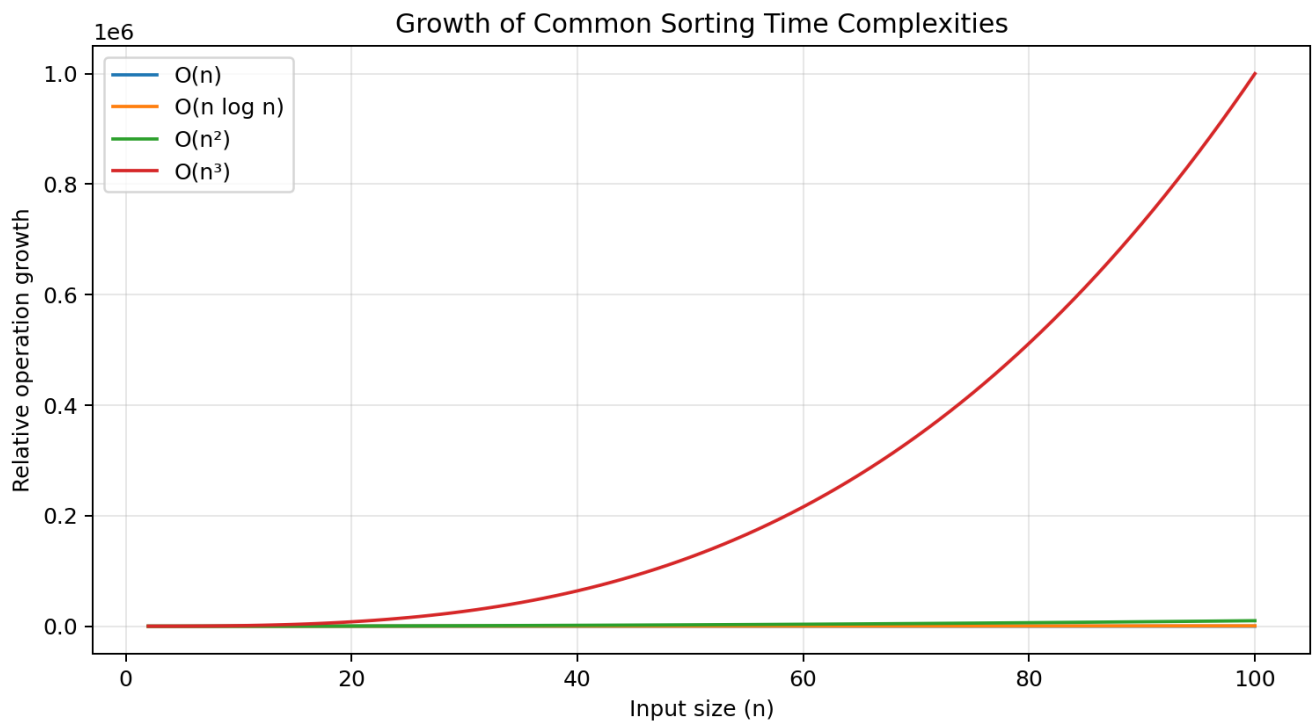
A concise exam/viva-ready reference covering the major comparison-based and non-comparison sorting algorithms.

Algorithm	Best	Average	Worst	Space	Stable?	In-place?
Bubble Sort	$O(n)$	$O(n^2)$	$O(n^2)$	$O(1)$	Yes	Yes
Selection Sort	$O(n^2)$	$O(n^2)$	$O(n^2)$	$O(1)$	No*	Yes
Insertion Sort	$O(n)$	$O(n^2)$	$O(n^2)$	$O(1)$	Yes	Yes
Merge Sort	$O(n \log n)$	$O(n \log n)$	$O(n \log n)$	$O(n)$	Yes	No**
Quick Sort	$O(n \log n)$	$O(n \log n)$	$O(n^2)$	$O(\log n)$ ***	No	Yes
Heap Sort	$O(n \log n)$	$O(n \log n)$	$O(n \log n)$	$O(1)$	No	Yes
Shell Sort	depends	$\approx O(n^{1.5})$ ****	$O(n^2)$ ****	$O(1)$	No	Yes
Counting Sort	$O(n+k)$	$O(n+k)$	$O(n+k)$	$O(n+k)$	Yes*****	No
Radix Sort	$O(d(n+k))$	$O(d(n+k))$	$O(d(n+k))$	$O(n+k)$	Yes*****	No
Bucket Sort	$O(n+k)$	$O(n+k)$	$O(n^2)$	$O(n+k)$	Yes*****	Usually no

* Standard selection sort is not stable. **Standard array merge sort needs $O(n)$ auxiliary memory. *** Recursion stack: $O(\log n)$ average, $O(n)$ worst. **** Shell-sort complexity depends on the chosen gap sequence. ***** Stable when implemented with a stable intermediate/output placement. ***** Bucket sort stability depends on the sorting method used inside buckets.

Complexity Growth Graph

The graph compares the growth rates of the most common complexity classes encountered in sorting. $O(n+k)$ and $O(d(n+k))$ depend on the value range/radix parameters, so they are not represented by a single universal curve here.



1. Bubble Sort

Repeatedly compares adjacent elements and swaps them when they are in the wrong order. After each pass, the largest unsorted element moves to the end.

- Best: $O(n)$ when an early-exit flag detects no swaps.
- Average: $O(n^2)$.
- Worst: $O(n^2)$, e.g. reverse-sorted input.
- Space: $O(1)$. Stable and in-place.

Note: Typical use: simple teaching examples or nearly sorted data; generally inefficient for large datasets.

2. Selection Sort

Finds the minimum element in the unsorted portion and swaps it with the first unsorted position.

- Best/Average/Worst: $O(n^2)$ comparisons.
- Space: $O(1)$.
- In-place.
- Standard implementation is not stable.

Note: Advantage: performs only $O(n)$ swaps, which can matter when writes are expensive.

3. Insertion Sort

Builds a sorted prefix one element at a time by shifting larger elements and inserting the current element into its correct position.

- Best: $O(n)$ for already/nearly sorted input.
- Average/Worst: $O(n^2)$.
- Space: $O(1)$.
- Stable and in-place.

Note: Excellent for small arrays and nearly sorted data; often used inside hybrid algorithms.

4. Merge Sort

Uses divide-and-conquer: split the array, recursively sort both halves, then merge the sorted halves.

- Best/Average/Worst: $O(n \log n)$.
- Space: $O(n)$ for the standard array implementation.
- Stable.
- Not in-place in the usual implementation.

Note: Reliable worst-case performance; especially useful when stable sorting is required.

5. Quick Sort

Chooses a pivot, partitions the array around the pivot, then recursively sorts the two partitions.

- Best/Average: $O(n \log n)$.
- Worst: $O(n^2)$ with consistently poor pivots.
- Average recursion space: $O(\log n)$; worst: $O(n)$.
- In-place in common implementations; not stable.

Note: Usually very fast in practice. Randomized or carefully chosen pivots reduce the chance of worst-case behavior.

6. Heap Sort

Builds a heap and repeatedly extracts the maximum/minimum element to place it in final position.

- Best/Average/Worst: $O(n \log n)$.
- Space: $O(1)$ auxiliary for array implementation.
- In-place.
- Not stable.

Note: Useful when guaranteed $O(n \log n)$ time and low extra memory are desired.

7. Shell Sort

Generalizes insertion sort by first comparing elements far apart using a gap sequence, then reducing the gap until it becomes 1.

- Complexity depends strongly on the gap sequence.
- Common textbook bounds vary; worst case is often given as $O(n^2)$.
- Space: $O(1)$.
- In-place and not stable.

Note: Faster than insertion sort on many medium-sized arrays, but less predictable than $O(n \log n)$ methods.

8. Counting Sort

Counts how many times each integer key occurs, then uses cumulative counts to place elements.

- Time: $O(n+k)$, where k is the key range.
- Space: $O(n+k)$ in the standard stable version.
- Not comparison-based.
- Can be stable.

Note: Very efficient when k is not much larger than n and keys are integers from a manageable range.

9. Radix Sort

Sorts keys digit-by-digit, commonly from least significant digit to most significant digit, using a stable subroutine such as counting sort.

- Time: $O(d(n+k))$, where d is number of digits/passes and k is the base/range per digit.
- Extra space typically $O(n+k)$.
- Can be stable.
- Not comparison-based.

Note: Useful for integers, fixed-length strings, and other keys with suitable digit representations.

10. Bucket Sort

Distributes elements into buckets according to value ranges, sorts each bucket, and concatenates the buckets.

- Best/Average: $O(n+k)$ under suitable distribution assumptions.
- Worst: $O(n^2)$ when many elements fall into one bucket and the internal sort is quadratic.
- Extra space: $O(n+k)$.
- Stability depends on implementation.

Note: Works well when values are approximately uniformly distributed over a known range.

How to Choose a Sorting Algorithm

Situation	Good choice	Reason
Nearly sorted / small input	Insertion Sort	Very fast when few shifts are needed.
Need guaranteed $O(n \log n)$	Merge Sort / Heap Sort	Both avoid quadratic worst-case time.
Fast practical general-purpose sorting	Quick Sort / hybrid variants	Excellent average performance and cache behavior.
Need stable sorting	Merge Sort / stable Insertion Sort	Preserves relative order of equal keys.
Integer keys with small range	Counting Sort	Can achieve $O(n+k)$.
Integer/fixed-digit keys	Radix Sort	Avoids comparison lower bound under suitable assumptions.
Uniformly distributed values	Bucket Sort	Can approach linear expected time.
Very limited extra memory	Heap Sort / Insertion Sort	$O(1)$ auxiliary space in standard forms.

Important exam points

- Comparison-based sorting has a general lower bound of $\Omega(n \log n)$ comparisons in the worst case for arbitrary inputs.
- Bubble, selection, and insertion sorts are $O(n^2)$ in the average/worst case, but insertion and optimized bubble sort can be $O(n)$ on already sorted input.
- Merge sort has predictable $O(n \log n)$ time and is stable, but the standard array implementation requires extra memory.
- Quick sort has $O(n \log n)$ average time but $O(n^2)$ worst case.
- Heap sort guarantees $O(n \log n)$ worst-case time with $O(1)$ auxiliary array space.
- Counting, radix, and bucket sorts are non-comparison sorts and can beat the comparison lower bound when their input assumptions hold.